
CAST 논문 진행 정리

무엇이 정해졌고 왜 그런지 · 근거 · 수치 · 남은 작업

이 파일은 "누가 무슨 작업을 했는지"의 기록이 아니라
"무엇이 정해졌고 무엇이 남았는지"만 적는다. 짧게 유지할 것.

[1] 한 줄 결론

KCI 투고 가능하다. 지금 필요한 건 새 실험이 아니라 논증 수정이다.

참고 논문 4편을 원문으로 대조한 결과, CAST는 평가 규모에서
이미 이들과 비슷하거나 앞선다. Caspian(학회 8쪽)보다는 확실히 앞선다.

[2] 참고 논문 4편 — 정체와 규모

Caspian	IEEE MASCOTS 2024, 학회 8쪽 공간+시간 동시(ILP), 예측기 없음, 용량 하드제약 3리전 / 합성 200건 / 40시간 / -33.2% 실제 K8s 컨트롤러 구현 (이게 강점)
CASPER	IGSC 2023, 학회 7쪽 공간 전용, 웹 요청 라우팅, 예측은 외부(CarbonCast) 사용 6리전 / 위키미디어 실트레이스 / 2022년 / 최대 -70% 용량 있음 (단 탄력적, 전역 서버수 K=500만 하드캡)
CarbonFlex	arXiv 2505.18357 (2025) — 정식 게재처 미확정, preprint 시간 전용 + 탄력적 스케일링, 단일 클러스터 KNN 사례기반 (신경망 예측기 아님) 실트레이스 3종 / AWS 실제 프로토타입 / -51~57% 용량 하드 · 온라인 제약
CICS	IEEE Trans. Power Systems (구글) 시간 전용, VCC(시간별 가상용량곡선)로 상한을 건다 구글 전체 fleet 실제 프로덕션 / 최고탄소 시간대 전력 -1~2%
CAST	8리전 / 합성 146,000건 / 2025년 1년 실측 / -56.85~65.93% LSTM 예측 + ILP 공간 + 시간 이동 시뮬레이션 (실배포 없음)

[3] 중요한 발견: "용량"은 더 이상 우리만의 것이 아니다

- 네 편 전부 어떤 형태로든 용량 제약을 다룬다.
- Caspian - 식(5) 하드 제약
 - CASPER - 탄력적 용량 + 전역 서버 상한
 - CarbonFlex - 하드 · 온라인, 게다가 용량 스윙(Fig.8)까지 했다
 - CICS - VCC로 시간별 상한, 프로덕션 배포

=> "용량을 고려한 연구가 없다"는 주장은 이제 못 쓴다.

그런데 아무도 안 한 게 하나 남아 있다.

"용량 제약이 없었다면 얼마나 줄었을 것을, 제약 때문에 잃었는가"
= 무제약 대비 반사실(counterfactual) 비교.
CarbonFlex조차 안 했다. 자기 알고리즘이 애초에 상한을 안 넘기 때문에
'무제약 시나리오' 자체를 돌려본 적이 없다.

CAST는 그걸 썼다.

이동 가능 작업의 53.6%가 되돌려짐
시간 이동 기여 9.07%p → 2.52%p
캘리포니아 동시실행 41건 = 상한의 3.4배, 연 867시간 초과

⇒ 기여 문구를 이렇게 바꾼다.

- (X) 용량 제약이 검토되지 않았다
- (O) 용량 제약이 시간 이동에서 앓아가는 몫이 정량화된 적 없다

[4] 지금 바로 고쳐야 할 것

순서대로. 1~4는 새 실험이 필요 없다.

1. Caspian 인용 추가 + 기여 #3 문구 좁히기 [문장 수정]
지금 본문에 Caspian이 한 번도 안 나온다.
그런데 문단 354가 "두 축 결합 시스템은 보고된 바 없다"로 끝난다.
Caspian이 정확히 그거다. 아는 심사위원이면 바로 잡는다.
2. 잘못된 직접인용 3개 수정 [필수]
직접인용 43개 중 37개는 원문과 정확히 일치한다. 3개가 문제다.
 - 문단 100, 335 : "our current work primarily focuses on exploiting spatial variations" → CASPER 원문에 이 문장 없음.
실제 문장은 "we focus on exploiting spatial characteristics that involves distributing workloads across regions..."
 - 문단 84 : "carbon footprint of a job can be up to 8x higher..."
→ CASPER 것으로 표기됐는데 원문에 없음. 출처 다시 찾을 것.
 - 문단 151 : "Cluster schedulers, such as Google's Borg..."
→ Sukprasert 것으로 표기됐는데 원문에 없음.심사에서 원문 대조당하면 가장 치명적인 항목이다.
3. 무릎점 자동 알파 검증 표 추가 [실험 불필요]
CASPER의 고정 $\alpha=0.5$ 를 비판해놓고 우리가 더 낫다는 표가 없다.
데이터는 이미 있다:
`load_balancer/02_프레임워크/results/summary.json`
핵심 한 줄:
auto의 평균 α 는 0.508로 고정 0.5와 사실상 같은데,
지연 6.7ms만 더 쓰고 탄소를 10.7%p 더 줄인다.
4. 기상 예보 누출을 7절 한계에 추가 [한 문장]
캘리포니아/텍사스/독일 LSTM 입력이 '미래 실측 날씨 슬라이싱'이다.
진짜 예보가 아니다. 문단 464가 스스로 경고하는데 한계 목록엔 없다.
코드로 확인되는 사실이라 발견되면 방어가 안 된다.
5. 6.2절 재서술 [해석만 변경]
LSTM이 1시간 지평에서 지속성 기준선에 0/8로 진다.
예측 품질의 기여는 전체 절감의 0.14%뿐이다.
숨기지 말고 발견으로 바꾼다:
"탄소 인지 스케줄링의 이득은 예측기 품질에 둔감하다"
그리고 6시간 지평은 8/8로 이기므로:
"공간 이동엔 예측이 거의 불필요, 시간 이동에만 필요하다"

6. 내부 메모 6건 + "근거 필요" 5건 정리 [반나절]
문단 111, 344는 이미 해소돼서 그냥 지우면 된다.
7. 용량 스왑 / 되돌리기 스크립트 복원 [하루]
6.4-6.5절 전체와 표 4,5,6이 이 결과 위에 서 있는데
스크립트가 레포에 없다. 재현 요구받으면 여기가 걸린다.
새 값을 구하는 게 아니라 논문에 적힌 수치를 재생산하는 것.
8. 스케줄러에 온라인 용량 제약 구현 [코드+재실행]
판단 필요. 아래 [6] 참조.

[5] 재현성 현황

되는 것 (직접 재실행으로 확인함)

run_cli.py 돌려서 -56.85% / -65.93% / SL0위반 0 재현됨
jobs.csv 는 SEED=42, N_DAYS=365 로 바이트 단위 동일 재생성
표1(예측 정확도) 전 셀 재계산 일치
LB -> 스케줄러 파이프라인 연결 실재 (carbon_region 주입)

안 되는 것

6.4 사후 용량 강제 / 6.5 용량 스왑 스크립트가 레포에 없음
LSTM 학습 스크립트 없음(코랩) -> 시간순 분할 검증 불가
탄소 데이터 출처(Electricity Maps)가 코드에 근거 없음

[6] 판단이 필요한 것 하나

스케줄러에 용량 제약을 넣을 것인가?

넣으면

헤드라인이 "59.37~65.93% 사이"라는 애매한 구간에서
하나의 숫자로 확정된다.
기여가 "측정"에서 "측정+해결"로 올라간다.
네 편 전부 용량을 다루는 상황에서 우리만 안 다루는 모양새를 면한다.

비용

코드 변경 자체는 작다.
compute_time_shift()의 후보 슬롯 루프에
(리전, 슬롯) 점유 카운터를 더해 상한 찬 슬롯을 빼면 된다.
CarbonFlex도 그리디 리스트 스케줄링이지 ILP가 아니다.
진짜 비용은 1년치 재실행 쪽이다.

중요한 점

어차피 [4]-7번(스왑 스크립트 복원)을 해야 한다.
그 하네스가 생기면 8번은 거의 공짜로 따라온다.
따로 계산할 일이 아니다.

[7] 이름 문제

같은 분야에 CASPER / Caspian / CarbonCast / CarbonFlex 가 이미 있다.
CAST는 CarbonCast와 너무 비슷하고, CASPER/Caspian과 앞 세 글자가 같다.

정정 논문 제목은 이미 잘 돼 있다. README만 낫았다.

논문 파일의 현재 제목 (문단 40-41)

국문: CAST: 리전 용량 제약을 고려한 탄소 인지 시공간 워크로드 스케줄링

영문: CAST: Carbon-Aware Spatio-Temporal Scheduling

국문 제목이 이미 "리전 용량 제약"으로 시작한다. 잘 잡았다.

낡은 건 README 쪽이다.

README: Carbon-Aware Scheduling and Traffic routing <- 옛날 이름

권고: 영문 풀어쓰기에서 한 단어만 바꾼다.

(현재) CAST = Carbon-Aware Spatio-Temporal

(변경) CAST = Capacity-Aware Spatio-Temporal

이유

Carbon-Aware 는 이 분야 논문이 전부 쓰는 말이라 차별점이 안 된다.

CASPER, Caspian, CarbonFlex, CarbonCast 다 탄소 인지다.

우리만 있는 건 "용량"이고, 국문 제목은 이미 그걸 앞세우고 있다.

영문도 맞추면 국문/영문/기여/결론이 한 단어로 꿰인다.

글자는 C-A-S-T 그대로라 바꿀 게 없다.

탄소가 빠지는 게 걱정되면 부제로 살리면 된다.

"Capacity-Aware Spatio-Temporal Scheduling for Carbon Reduction"

[8] 참고 논문에서 가져와도 되는 것 / 안 되는 것

가져와도 되는 것 (정상적인 학술 관행이다)

- 절 구성 순서 (문제정의 -> 형식화 -> 알고리즘 -> 평가 -> 결론)
- 그림의 종류 (아키텍처 그림, 입력 탄소 트레이스 그림, 시간별 배분 그림, 가중치 스윙 그림)
- 평가 설계 (가중치 스윙, 단계별 어블레이션)
- 지표 정의식 ($CS = 100 * (C - C^*) / C$ 같은 것). 인용만 달면 된다.
- 표기 규약, 한계를 서술하는 방식

가져오면 안 되는 것

- 문장. 번역해서 쓰는 것도 안 된다.
카피킬러는 한국어 문자열을 잡으므로 번역문은 검사에 안 걸리지만, 원문을 아는 심사위원에게 걸리면 그게 더 나쁘다.
- 그들의 수치를 우리 것처럼 쓰는 것
- 그림 이미지 자체

실무 규칙

구조는 베끼고 문장은 직접 쓴다.

그리고 베낀 구조는 밝힌다.

"평가 구성은 Caspian[x]의 방식을 따랐다"라고 쓰면

그건 표절이 아니라 비교가능성 확보다. 오히려 점수를 탄다.

[9] 절 구조를 바꾸기로 함

(기존 7절)	(새 구조)
1. 서론	-> 1. Introduction
2. 배경 지식	-> 1 또는 2로 흡수 (조사중)
3. 풀고자 하는 문제	-> 3의 첫 소절로 흡수 (조사중)
4. 기존 방법 + 한계	-> 2. Related Work
5. 제안하는 방법	-> 3. CAST
6. 검증	-> 4. Experiments and Analysis
(없음)	-> 5. Discussion and Limitations <- 새로 생김
7. 결론	-> 6. Conclusion
	Abstract / References 추가

확인할 것

기록상 7절 구조는 교수님이 정하신 고정 사항이었다.
바꾸는 게 확정인지 먼저 확인할 것.

바꾸면 좋은 점

5. Discussion and Limitations 가 독립 절이 되는 게 크다.
지금은 한계 4가지가 결론 안 문단 848 한 곳에 뭉쳐 있다.
CAST는 정직하게 밝힐 한계가 많은 논문이라
(기상 예보 누출 / LSTM 기여 0.14% / 시뮬레이션 / 용량 미구현)
이걸 펄 자리가 생기는 건 방어에 유리하다.
약점을 먼저 말하는 논문이 심사에서 덜 맞는다.

주의

절이 하나 늘면 분량도 는다.
KCI 분량 제한을 먼저 확인하고 시작할 것. (조사중)

[9-1] 옮기는 표 (문단번호 -> 새 절)

문단번호는 word/document.xml 의 870문단 기준.

0-39	삭제. 논문 내용이 아님 학회 후보 목록이랑 클로드 세션 링크가 파일 안에 들어있다. 투고 전에 무조건 지울 것.
40-41	제목. 유지.
(없음)	-> Abstract : 새로 써야 함. 지금 초록이 아예 없다.
57-90	-> 1. Introduction 그대로
92-119	-> 2. Related Work (지금은 서론에 있음) 옮기고, 서론엔 1~2문장 요약만 남긴다
122-135	-> 1. Introduction 끝 기여 4개 + 헤드라인 수치
138-147	-> 3. CAST 서두 리전 특성. 6.1과 겹치니 합칠 것
149-152	-> 2. Related Work 작업 지연허용도 분류
154-167	-> 2. Related Work 예측 가능성
222-233	-> 3. CAST 첫 소절 문제정의+제약. 5.2의 식(2)-(5)와 사실상 중복이라 합친다
235-240	-> 1. Introduction 끝 결정의 구조. 한두 문장으로 압축
242-247	-> 5. Discussion "다루지 않는 것" = 한계의 일부
330-355	-> 2. Related Work 기존방법+한계 통째로

426-518	-> 3. CAST	제안하는 방법 통째로. 거의 그대로
564-838	-> 4. Experiments	검증 통째로. 소절 번호만 6.x -> 4.x
843-847	-> 6. Conclusion	압축
848	-> 5. Discussion	한계 4가지. 펼쳐서 확장
850	-> 5. Discussion	향후과제 3가지 (Future Work 소절)

168-219 / 248-327 / 356-423 / 519-561 / 852-869
전부 빈 문단 덩어리. 삭제.

[9-2] 새로 생기는 5. Discussion and Limitations 에 넣을 것

지금 한계가 결론 안 문단 848 한 곳에 1문장씩 뭉쳐있다.
독립 절로 퍼려면 이만큼 필요하다.

1. 기존 한계 4가지를 각각 1문장 -> 1문단으로
(왜 생기는지 + 결과를 어느 방향으로 틀었는지)
2. 기상 3리전 미래실측 누출 <- 5번째 한계로 신설. 지금 없다.
3. 65.93% / 59.37% 두 값 사이를 왜 하나로 못 냈는지 논의
(지금은 결론에 숫자만 있고 설명이 없다)
4. 예측 정확도가 결과에 0.14%만 영향 준다는 발견을 여기로 올림
5. 3.4절 "다루지 않는 것"(PUE/냉각, 내재탄소, 전송비용)을 서두로
6. Sukprasert 차별화 논증 마무리 (문단 344 메모가 미완성 표시)
7. 향후과제 3가지는 Future Work 소절로 분리

[9-3] 분량

지금 본문 약 38,500자 (공백포함, 영어 원문인용 포함)
국내 컴퓨터공학 KCI 학술지는 보통 8~12쪽
-> 대략 20,000~28,000자가 안전권으로 추정

지금 1절과 4절에 영어 원문 인용이 길게 늘어져 있는데
이걸 각주나 참고문헌으로 빼면 실질 분량은 꽤 준다.
안 빼면 초과 가능성 있음.

* 이 숫자는 추정이다. 목표 학회지 규정은 아직 확인 안 됐다. (조사중)

[10] 실제 KCI 논문은 어떻게 생겼나

실제 KCI 등재 논문지 3편을 원문으로 확인함.

- 1) 한국산학기술학회논문지 2020, 8쪽
CNN-LSTM 캠퍼스 전력 예측
그림6 표6 참고문헌8편, 국문요약 806자
실험: 캠퍼스 1년치 전력, 기존 시계열모델과 비교
- 2) 한국컴퓨터정보학회논문지 2021, 11쪽
HPC 에너지 절감 (본문 영문)
그림11 표3 참고문헌22편, 국문요약 434자
실험: NPB 벤치마크, 단일 노드, 전력 6% 감소

3) 디지털콘텐츠학회논문지 2021, 8쪽
배치 작업 스케줄링
그림7 표10 참고문헌20편, 국문요약 520자
실험: 시뮬레이션 예시 (job 몇 개 단위. 규모 작음)

* 레포에 있던 "드리프 패널티" 논문은 KCI 논문지가 아니라
2쪽짜리 학술대회 발표논문집이다. 표본으로 쓰면 안 된다.

중요: 우리 문제는 부족이 아니라 과잉이다

분량 KCI 8~15쪽 <-> 우리 38,500자 (훨씬 김)
그림+표 KCI 9~17개 <-> 우리 그림4 표6 = 10개 (적정)
참고문헌 KCI 8~22편 <-> 우리 그만큼 있음 (적정)
실험규모 KCI 단일 벤치마크/시뮬레이션 예시 수준
<-> 우리 8리전 146,000건 1년 (명백히 과함)

실험 규모가 과한 건 흠이 아니라 강점이다. 줄이지 마라.
대신 두 가지를 해야 한다.

1. 서술을 압축해서 8~12쪽에 맞춘다.
영어 원문 인용을 각주/참고문헌으로 빼는 게 제일 크다.
2. "왜 이렇게 큰 규모로 했는가"를 한두 문장으로 정당화한다.
지금은 그냥 크게 했다고만 돼 있다.

KCI는 "엄밀한 반박"보다 "제안-검증의 완결된 서사"를 본다.
지금 우리 4.4절의 Sukprasert/Attenni 반박은 국제 학회급인데,
KCI에서는 그게 오히려 길다는 뜻이다. 압축 대상 1순위.

[10-1] 한국 논문 관례 (3편 모두 일치)

표 제목은 표 "위"에, 그림 캡션은 그림 "아래"에
국문/영문 제목 · 저자 · 요약 병기 (요약 400~800자)
참고문헌은 [1] 대괄호 번호 인용
저자 소속에 직급 명시, 교신저자 * 표기
Received / Revised / Accepted 날짜 명시
학회에 따라 논문 말미에 저자 약력(사진+한 단락)

투고할 학회 양식을 먼저 받아서 확인할 것.

[11] 학회지 4곳 규정 (공식 규정 + 실제 게재논문 3편씩 확인)

	TKIPS	KTCP	JKIIT	JKIICE
게재료 기본	20만	-	15만(6면)	20만(6쪽)
쪽 초과	5만/쪽	1.5~5만/쪽	3만/면	3~4만/쪽
심사료	5만(상충)	5만	5만	4만
영문초록	규정없음	100~200단어	100~200단어	130~160단어
국문초록	규정없음	300~500자	-	-
키워드	3~5	4~6	미명시	4~5
참고문헌(표본)	10/12/14	11	9/30/17	9/9/9
그림+표(표본)	7/5/18	확인못함	16/16/7	7/9/7

* TKIPS = 구 KTCCS. 2024.1.8 통합됨.
* 4곳 다 2단 조판, 국문+영문 초록 필수

- * 4곳 다 쪽수 상한이 "없다". 대신 초과분 누진 게재료로 누른다.
- * 표절 % 기준은 4곳 다 공식 규정에 없음.
"카피킬러 몇 % 이하"라는 통설은 근거를 못 찾음.
- * JKIIICE는 2026.4.9 개정으로 생성형AI 사용 고지 의무가 생김.

분량 초과는 벽이 아니라 돈이다

쪽수 상한이 없으니 길어도 낼 수는 있다. 돈이 더 들 뿐이다.
예) JKIIICE에 12쪽으로 내면
 $20\text{만} + (7\sim 8\text{쪽 } 2\text{면} \times 3\text{만}) + (9\sim 12\text{쪽 } 4\text{면} \times 4\text{만}) = 42\text{만원}$
줄여야 할 이유가 심사가 아니라 비용이다.
인용 장치를 각주로 빼면 12,910자가 빠진다. 그것만 해도 크다.

[11-1] ** 정정: 절 구조는 5단이 표준이다 **

앞에서 "5. Discussion and Limitations 독립 절이 좋다"고 적었는데
실제 표본 12편(4학회 x 3편) 중 그렇게 한 논문이 0건이다.

실제로 제일 흔한 구성 (5단)

1. 서론
2. 관련 연구
3. 제안 기법
4. 실험 및 분석
5. 결론

6단을 쓰는 경우도 있는데, 추가되는 절이 "논의및한계"가 아니라
"배경(Background)"을 관련연구에서 떼는 식이었다 (TKIPS 1건).

그래서 권고를 바꾼다.

- 5단으로 간다.
- 한계는 독립 절로 빼지 말고 "5. 결론" 안에서 확장한다.
지금도 문단 848(한계4가지) 850(향후과제3가지)이 결론에 있다.
자리를 옮기지 말고 거기서 늘리면 된다.
- 국내 관행으로는 마지막 절 제목을 "결론 및 향후 연구"로
하는 경우가 많다. 그렇게 하면 한계/향후과제가 자연스럽다.

기상 예보 누출 같은 한계는 여전히 반드시 넣어야 한다.
자리가 독립 절이 아니라 결론 안일 뿐이다.

[12] 투고 전 반드시 지울 것

문단 0~39 가 논문 내용이 아니다. 작업 메모다.

- KCI/SCIE 학회 후보 목록과 게재료 메모
- 클로드 대화 공유 링크 2개

그대로 두고 투고하면 안 된다.

[13] 결과를 어떻게 낼 것인가 (베이스라인)

고민: 다른 논문은 벤치마킹하는데 우리는 비교 대상이 없다.

생각을 바꿔야 한다.
베이스라인은 남의 코드를 돌리는 게 아니라
남의 "정책"을 우리 데이터 위에서 재현하는 것이다.
이게 표준이고 심사에서 문제되지 않는다.

B0	홈 리전 즉시 실행	탄소 무시	있음
B1	고정 $\alpha = 0.5$	CASPER 정책	있음 (summary.json)
B2	고정 $\alpha = 1.0$	탄소만 보는 탐욕	있음
B3	시간 이동 단독	Sukprasert 주장	없음
OR	완전 지식 오라클	이론 상한	있음 (9,647.1 kg)
--	CAST	제안 기법	있음

여섯 개 중 다섯 개가 이미 있다. 이름만 안 붙어 있었을 뿐이다.

B3만 한 번 더 돌릴 것을 권한다.
홈 리전에 그대로 두고 시간 이동만 적용하는 것.
scheduler.py 에 mode 분기가 이미 있으니
simple_lb_timeshift 하나 추가하면 끝. 네 줄짜리다.

왜 B3가 중요한가
Sukprasert 논문의 주장이 정확히
"시간 이동 단독은 이득이 제한적이다" 이고
우리 논문 전체가 그 논문과 대화하고 있다.
B3가 있으면 "우리 데이터에서도 그렇더라/아니더라"를
직접 말할 수 있다. 지금은 인용으로만 말하고 있다.

용량 스왑 하네스 복원할 때 같이 돌리면 된다. 따로 할 일 아니다.

[14] 가정이 많다는 걱정에 대해

Caspian 이 실제 쿠버네티스에 돌린 건 sleep 명령이다.

- k3d = 로컬에 띄운 가짜 클러스터
- 작업 200건 전부 BusyBox 에 sleep 건 것
- 각주9: 사용률을 실측 안 하고 요청량 비례로 가정
- 전력효율 $\rho = 100/200$ 상수 가정 (그냥 정한 값)
- 워크로드 200건도 합성 (Poisson, $U(1,5)$)
- 탄소 데이터 40시간 한 구간

우리 $P = 1\text{kW}$ 가정과 Caspian 의 ρ 가정은 같은 종류다.
우리가 더 나쁘지 않다. 탄소는 오히려 우리가 1년치 실측 8리전이다.

진짜로 우리보다 앞선 건 둘뿐이다.
CarbonFlex (AWS 실배포 + 실트레이스 3중)
CICS (구글 프로덕션)
이 둘은 못 따라간다. 따라갈 필요도 없다.

진짜 약점은 하나다

"가정이 많다"가 아니라 "합성 워크로드 파라미터에 근거가 없다"이다.
문단 56에 이미 (근거 필요) 메모가 붙어 있다.

- 리전당 하루 50건
- k 비율 3:2:1:1:3

해결은 실트레이스로 갈아타는 게 아니다.
공개 트레이스(Azure, Alibaba, Google cluster trace)의
발표된 통계를 인용해서

"우리 생성기는 이 분포를 따랐다"로 만들면 된다.
실험을 다시 할 필요 없이 인용만 붙이는 일이다.

바로잡을 것
각 나라 12개는 job 개수가 아니라 동시 실행 상한이다.
작업은 리전당 18,250건이고 동시에 도는 게 최대 12건이다.
게다가 12는 임의값이 아니라
기준 배경의 최대 동시실행수 x 1.2 x 여유율 0.8 로 나온 값이다.
Caspian 의 "GPU 32개"보다 근거가 낫다.

[15] 참고문헌 적는 방식

지금처럼 문장 아래에 바로 적는 방식이 맞다. 계속할 것.
어제 없는 인용문 3개가 나온 게 그 습관을 안 지킨 문단들일 것이다.

- 두 가지만 바꾼다.
1. 각주로 옮긴다.
지금은 본문 문단 안에 섞여 있어 분량 계산이 안 되고
지울 때 실수가 난다. 워드 각주면 일괄 처리된다.
 2. 번호는 맨 마지막에 붙인다.
중간에 [1][2] 붙이면 순서 바뀔 때마다 다 들어진다.
지금처럼 URL 로 달아두고 두고 직전에 한 번에 변환.

투고 직전에 인용 대조 스크립트 한 번 돌릴 것.

[16] 분량 – 문장을 줄이는 게 아니라 인용을 내리면 된다

본문 총	38,531자
인용 장치	12,910자 (33.5%)

빼면	25,621자
----	---------

절별	총	인용뺀것	인용비중
1 서론	8,248	3,608	56%
2 배경지식	4,672	2,088	55%
4 기존방법	6,181	3,262	47%
5 제안방법	8,069	6,570	19%
6 검증	7,783	7,086	9%

"주저리주저리 길다"고 느낀 게 서론/배경지식/기존방법인데
그 절들은 절반 이상이 영어 원문 인용이다.
문장을 줄이는 게 아니라 인용을 각주로 내리면 된다.

5절과 6절은 인용이 10~20%뿐이라 실제 서술이다.
거기는 건드리지 말 것. 방법과 검증이 논문의 몸통이다.

[17] alpha 표 완성 – 2026-09-16, 10초 걸림

로드밸런서 재실행 없이 배정 파일 7개를 현행 회계로 다시 계산.

alpha_auto 가 논문 표2 ②(12,609.8)와 정확히 일치함.

즉 이 표는 표2와 같은 회계 위에 있다. 그대로 논문에 넣어도 된다.

run	탄소(kg)	절감	평균지연	p95	호비율	드롭
baseline	29,225.6	-	0.0 ms	0	100.0%	0
alpha=0.00	29,225.4	0.00%	0.0 ms	0	100.0%	0
alpha=0.25	25,989.6	11.07%	1.5 ms	12	88.2%	0
alpha=0.50	15,718.6	46.22%	29.5 ms	137	59.0%	0
alpha=auto	12,609.8	56.85%	36.2 ms	137	56.3%	0
alpha=0.75	10,396.5	64.43%	62.6 ms	204	39.6%	0
alpha=1.00	10,004.4	65.77%	114.4 ms	238	13.0%	0

<- 평균 0.508

무릎점 논증 – 지연 1ms 더 쓸 때 줄어드는 탄소

0.25 -> 0.50	지연 +28.0 ms	탄소 -10,271.0 kg	366.6 kg/ms
0.50 -> auto	지연 + 6.7 ms	탄소 - 3,108.8 kg	464.9 kg/ms <- 최대
auto -> 0.75	지연 +26.4 ms	탄소 - 2,213.3 kg	83.9 kg/ms <- 급락
0.75 -> 1.00	지연 +51.8 ms	탄소 - 392.1 kg	7.6 kg/ms

auto 가 한계 수익 최대 지점이고, 바로 다음 구간에서 5.5배 떨어진다.
이게 "무릎에 선다"의 근거다.

논문에 쓸 한 문장

고정 alpha=0.5 15,718.6 kg 절감 46.22% 지연 29.5 ms
auto (평균 0.508) 12,609.8 kg 절감 56.85% 지연 36.2 ms
-> 평균적으로 같은 가중치를 쓰면서, 지연 6.7 ms 만 더 쓰고
탄소를 3,108.8 kg (10.64%p) 더 줄인다.

주의: "탄소 최저"라고 쓰면 안 된다.

alpha=0.75 가 10,396.5 로 더 낮다.
"무릎에 선다"로만 써야 한다.

재계산 스크립트 (잃어버리지 말 것)

scratchpad/alpha_table.py (탄소)
scratchpad/alpha_lat.py (지연 · 한계수익)
둘 다 20줄 내외. 레포 코드는 안 건드릴. 배정 CSV만 읽는다.
다시 만들려면: 배정 CSV의 assigned 열을 scheduler.schedule_job 에
mode='carbon_lb_immediate' 로 통과시켜 carbon_emitted 를 합하면 된다.

[18] 시뮬레이션 걱정에 대한 답 – 방어 논리 확보

Sukprasert(EuroSys 2024)는 실배포 없이 트레이스 분석만으로
EuroSys 에 실렸다. EuroSys 는 우리 목표인 KCI 보다 훨씬 높다.

중요: 그 논문은 "실배포 안 한 게 한계"라고 안 쓴다.

아예 자기를 시스템 논문이 아니라 측정/분석 논문으로 포지셔닝한다.

- 제목부터 "On the Limitations of..."
- 초록: "we conduct a detailed data-driven analysis to understand the benefits and limitations..."
- 규모를 기여로 내세움: "123 regions worldwide... over

three years (2020-2022)"

- 선행연구를 이렇게 비판: "most analyses focus on a specific setting... using a small number of geographical regions"

즉 논법이 이렇다.

"123리전 x 3년 규모의 비교는 실패포로는 애초에 불가능하다.
그 규모 자체가 이 논문의 기여다."

CAST 에 그대로 쓸 수 있다. 논리 구조가 똑같다.

우리 핵심 실험은 용량 제약을 일부러 끄고 넘치게 두는 것이다.
이건 실제 프로덕션에서 할 수 없는 실험이다.
구글이 VCC 를 켜줄 때 무슨 일이 생기는지 재보겠는가? 못 한다.

4.4절에 이 한 문장을 넣을 것.

"본 절의 비교는 용량 제약을 의도적으로 해제한 조건을 필요로 하므로
운영 중인 시스템에서는 수행할 수 없다."

그리고 5절 한계에서 "시뮬레이션이라 한계"라고 사과하지 말 것.
방법론 선택으로 쓸 것. Sukprasert 인용 붙이면 된다.

[19] 수도코드 형식 – 참고 논문 관행

Caspian (IEEE 2단)

Alg1 9줄 / Alg2 15줄 / Alg3 7줄, 전부 1단 폭
입력을 제목에 씀: "Optimizer(M, N)"
출력은 코드 마지막 줄에 "Output: y" 로 인라인

CarbonFlex (ACM 2단)

Alg1 16줄 / Alg2 6줄 / Alg3 10줄, 전부 1단 폭
"Input: ... Output: ..." 헤더를 코드 앞에 명시

두 논문 공통 관행

- 블록당 10~20줄, 1단 폭. 2단 전체를 쓰는 긴 알고리즘은 없다.
- 알고리즘 박스 안에 식 번호를 안 쓴다.
박스 앞뒤 설명 문단에서 "식 (14)에 따라..." 로 산문 인용한다.
- 설명 산문이 코드의 3~5배 분량.

CAST 적용

블록 2개면 충분하다.
Algorithm 1 지금 방식 (사전 예약형) 짧게
Algorithm 2 새 방식 (슬롯마다 재평가) 15줄쯤
Input/Output 헤더는 CarbonFlex 방식으로 명시하는 게
KCI 심사에서 더 명확하다.

[20] 식을 6개에서 3개로 줄인다

아까 식 (12)~(17) 여섯 개를 잡았는데 너무 많다.
이미 (1)~(11)이 있는데 여섯 개를 더 없으면 읽는 사람이 지친다.

남길 것 3개

가용 자리	$a_r(t)$	용량의 정의라 필수
희망 집합	$P_r(t)$	"지금 돌고 싶은 작업들"
우선순위	$\rho_j(t)$	자리 부족할 때의 규칙

빠는 것 (수도코드에 한 줄씩 녹임)

대기 집합 $Q(t)$ / 윈도우 $W_j(t)$ / 선택 절차 $S_r(t)$

* $W_j(t)$ 는 이미 5.5절 문단 496에 정의돼 있어서 새 식이 필요 없다.

그리고 좋은 발견

가용 자리 식은 이미 논문에 있는 식 (3)을 뒤집은 것이다.

식 (3) $\{j : r(j)=r, \tau_j \leq t < \tau_j+d_j\} \leq \text{floor}(\eta \cdot \text{cap}_r)$

새 식 $a_r(t) = \text{floor}(\eta \cdot \text{cap}_r) - \{ \dots \}$

같은 말이다. 그래서 이렇게 쓸 수 있다.

"5.2절에서 식 (3)으로 선언했으나 구현하지 않았던 제약을
이 알고리즘이 실제로 강제한다."

진짜 새 식은 두 개뿐이다. 부담이 거의 없다.

[21] 회계가 두 개 있다 — 알아둘 것

summary.json 은 옛날 버전이 아니다. (내가 처음에 잘못 말했음)

CSV 의 carbon_g 합계와 7개 행 전부 정확히 일치한다.

로드밸런서 회계 : 실행 구간을 적분	-> summary.json, CSV
스케줄러 회계 : 정시 단위 빈 평균	-> 논문 표2, alpha 표

둘 다 맞는 계산인데 방식이 다르다. 차이는 최대 0.5%.

alpha=1.0 행만 51kg(0.5%) 차이나는 이유:

alpha=1 에서 작업 실행시간 중앙값이 78.5초로 짧아진다(0.75는 91.4초).

짧은 작업은 "한 시간 안에서 어디 놓이냐"에 민감해서 차이가 커진다.

오류가 아니다.

논문은 스케줄러 회계로 통일돼 있다. 표2도 alpha 표도 그것.

그러니 alpha 표에는 재계산한 숫자를 쓴다. ([17] 참고)

주의: 나중에 누가 재현성 확인하려고 summary.json 을 열면

같은 불일치를 또 만난다. 어느 회계를 쓴 숫자인지 각주로 밝히거나

summary.json 을 스케줄러 회계로 다시 써두는 게 낫다.

[22] 시뮬레이션 방어 — 논법을 바꿨다

찾아본 결과 ** "프로덕션에서 못 하는 실험이다" 라는 정형 인용문은

확인되지 않았다. ** Sukprasert 에도 그런 절이 없고(규모 논법만 있음),

카오스엔지니어링 논문도 원문 확인 실패.

대신 더 나은 카드 두 개가 나왔다.

(1) KCI 스케줄링 논문은 정당화조차 안 하고 통과한다

디지털콘텐츠학회 2021 (윤준원 · 송의성, 배치 작업 스케줄링)

- 작업 7개짜리 수작업 예시 표로 세 기법 비교. 시뮬레이터도 안 씀.

- 결론에 "향후 실제 클러스터에 반영하여 검증할 필요가 있다" 한 줄.

- 왜 실패표를 안 했는지 정당화하는 문장이 아예 없다. 그래도 게재됨.

정보처리학회 2019, 한국시뮬레이션학회 2011 도 시뮬레이션 전용.

=> "정당화가 반드시 필요하다"는 전제 자체가 과장된 걱정이었다.

- (2) 반사실 논법 — 이게 위험 논법보다 강하다
 (버릴 것) "프로덕션을 일부러 터뜨릴 수 없다"
 -> "테스트 클러스터에서 하면 되지 않나"로 반박된다.
 (쓸 것) "같은 작업 집합에 용량을 강제한 경우와 안 한 경우를
 나란히 놓아야 하므로, 한 시스템으로는 원리상 불가능하다"
 -> 반박이 안 된다. 한 시스템이 같은 작업에 두 정책을
 동시에 적용할 수 없다. 논리적 불가능이라 인용도 필요 없다.

4.5절에 이 문장으로 넣을 것.

그리고 5절 한계에서 "실배포 아님"이라고 사과하지 말 것.

[23] 2026-09-16 마무리 — 지금 어디까지 왔나

오늘 정해진 것

- 논문의 주인공을 바꿨다.
 (전) 두 축을 결합한 시스템을 만들었다
 (후) 파레토 무류점으로 가중치를 자동 결정한다 <- 기여 1
 용량이 시간 이동의 몫을 먹는다는 것을 잰다 <- 실험의 발견
 이유: "두 축 결합"은 Caspian 이 이미 했고 실제 K8s 에서 했다.
 무류점은 우리가 만들었고 구현돼 있고 검증표까지 있다.
 선행 3편(CASPER/CICS/Caspian)이 "가중치를 어떻게 정할지 모르겠다"고
 직접 써놨다. 그 숙제를 우리가 푼다.
- 절 구조는 5단. KCI 표준이다. ([11-1])
 1 서론 / 2 관련연구 / 3 제안기법 / 4 실험및분석 / 5 결론
 한계는 독립 절로 빼지 말고 결론 안에서 늘린다.
- 용량 인지 온라인 시간 이동을 구현하기로 했다.
 기존 숫자가 안 바뀌고 표2에 행 하나가 추가되는 구조라 안전하다.
 우선순위는 순수 EDF(남은 여유 적은 것부터). 82가 내 식의 버그를 잡았다.
- 시뮬레이션은 사과하지 않는다. 반사실 논법으로 쓴다. ([22])

오늘 실제로 끝난 것

alpha 표 완성	숫자 확보, 표2와 일치 확인 ([17])
그림 1 수정	곡선 vs 값 1개 대비 + 각주 추가
백업	paper/백업/20260916-0100_그림수정/
목차.txt	절별로 뭘 쓸지 다 적음

다음에 할 것 — 실험 없이 되는 것부터

- | | | |
|-----------------------|----------------------|------------|
| 1 없는 인용문 3개 수정 | 문단 84, 100, 151, 335 | 최우선 |
| 2 초록 쓰기 | 국문 300~500자 + 영문 | |
| 3 기여 순서 바꾸기 | 1절 | |
| 4 Caspian 관련연구 추가 | 2.3절 + 구조차용 문장 | |
| 5 alpha 표 + 파레토 그림 삽입 | 4.3절 | |
| 6 기상 예보 누출 한계 추가 | 3.3절 + 5절 | |
| 7 소절 제목에 alpha 넣기 | 3.4 / 4.3 | |
| 8 내부 메모 · 근거필요 정리 | 11곳 | |
| 9 문단 0~39 삭제 | 학회메모 + 클로드 링크 | |

돌려야 하는 것

10 B3 시간 이동 단독	4줄, 16초
11 용량 인지 구현	200줄, 16초
12 용량 스왑 스크립트 복원	시간 미측정

* 11번의 simulator.py 구조 개편(40~80줄)은 SimPy 를 걷어낼지 결정이 필요하다. 조원 상의 사항.

아직 안 정한 것

투고 학회 (TKIPS / KTCP / JKIIIT / JKIIICE 중)
-> 양식 파일을 먼저 받아야 분량 · 초록 · 저자정보 기준이 확정된다
fig_architecture_캡션.md 는 예전에 버린 그림을 설명하고 있음. 다시 써야 함

작업 문서 두 개

정리.txt 무엇이 정해졌고 왜 그런지 (근거 · 수치 · 결정)
목차.txt 각 절에 무엇을 쓸지 (집필용)

[24] 용량 인지 구현 — 검증 기준 (결과 보기 전에 못박음)

2026-09-16. 구현은 82, 코드리뷰는 ac, 숫자 검증은 1b.
사용자 지시: 짠 사람이 검증하면 자기 가정에 맞춰지므로 분리한다.

통과해야 하는 것 — 하나라도 여기면 구현이 틀린 것

- C1 마감 위반 = 0
근거: 문단 636 "작업 146,000건은 전량 마감 이내에 처리되어 마감 위반은 발생하지 않았다". 이게 논문 셀링포인트다.
- C2 처리 작업 = 146,000 (드롭 0)
- C3 총 탄소가 두 값 사이에 있어야 한다
무계약 9,958.2 kg $\leq X \leq$ 사후강제 11,873.0 kg
- 무계약보다 낮으면 불가능. 계약을 더했는데 더 좋아질 수 없다.
- 사후강제보다 높으면 알고리즘이 잘못됐다.
사후강제는 일부러 멍청하다(원래 시각으로 되돌리기만 하고 재배치 안 함).
제대로 만든 온라인이 그보다 나쁘면 구현 오류다.
- C4 최대 동시 실행 ≤ 12
강제 편입(마감 임박)이 있으면 그만큼만 초과 허용.
강제 편입이 0인데 12를 넘으면 장부가 틀린 것.
- C5 $k=3,4,5$ 는 delay = 0
자연 예산이 1시간 미만이라 시간 이동 대상이 아니다.
87,712건. 이들이 밀리면 C1 이 깨진다.
- C6 delay > 0 인 작업 수 $\leq 58,288$
시간 이동 가능한 작업은 $k=1,2$ 뿐이다. (문단 568)
- C7 탄소 회계가 기준과 일치
용량을 무한대로 주고 즉시 실행시키면 비교군2 12,609.8 kg 이 나와야 한다.
안 나오면 회계 방식이 달라 표2와 비교 불가.

검증 방법 — 코드의 자기 보고를 믿지 않는다

최대 동시 실행을 capacity.py 의 장부에서 읽지 말고,
결과(scheduled_start, duration, region)에서 다시 계산한다.
본문 § 6.4 정의 그대로: 모든 작업의 시작 · 종료 시각에서 누적합.
장부가 정시만 샘플링하는데 본문은 연속 시각이므로
두 값이 다를 수 있다. 다르면 그게 발견이다.

검증 스크립트: scratchpad/verify_cap.py

[25] 용량 인지 1차 검증 결과 — 2026-09-16

구현 82 / 코드리뷰 ac / 숫자검증 1b 로 분리해서 진행.

** 역할 분리가 실제로 효과가 있었다. ac 가 코드에서 찾은 문제를
1b 가 숫자로 확인했고, 1b 가 짤 코드의 버그를 ac 가 찾았다. **

알고리즘 자체는 맞다

C=12 온라인 용량 인지 결과 10,898.9 kg
무제약 9,958.2 <= 10,898.9 <= 사후강제 11,873.0 (C3 통과)
사후강제 대비 구간의 50.9% 를 회수했다.
마감 위반 0 (C1 통과) / 처리 146,000 전량 (C2 통과)
실행 시간 7초

-> 논문에 쓸 수 있는 결과다. 표2에 ④행으로 들어간다.
"실현 가능한 절감은 59.37~65.93% 사이" 가
"온라인 용량 인지로 62.7% 를 달성" 으로 바뀔 수 있다.
(10,898.9 / 29,225.6 = 62.71% 감소)

남은 버그 2개 (82에게 넘김)

- 짧은 작업이 밀린다 (C5 · C6 실패)
k=3,4,5 중 1,420건이 지연됨(0이어야 함). 총 지연 59,708 > 58,288.
원인: 분수 시각 대 정시의 기준점 차이.
기존 코드는 t_now = submit_time(분수) 기준이라 후보에 "지금 당장" 이 포함.
롤링 루프는 t 가 정시라 lo = ceil(frac) = 1 이 되어 "제출 즉시" 가 빠진다.
그리고 즉시실행 우회 조건 `(deadline-duration) <= submit\_time` 이
L>0 이므로 항상 거짓이어서 모든 작업이 큐로 들어갔다.
- 용량 장부가 § 6.4 정의와 다르다 (ac 지적 -> 실측 확인)
장부 자기보고 CAL 16 vs § 6.4 정의로 재계산 CAL 18
1시간을 안 걸치는 작업들이 같은 구간에 겹칠 때 장부가 못 본다.
리전별 최대: DE 12 FR 16 IN 7 JP 12 KR 12 CAL 18 NY 14 TEX 12
상한 초과 시간: FR 22h, CAL 8h
-> 용량 체크를 연속 시각 기준으로 해야 한다.

검증 기준 C7 은 잘못 설계됐다 (1b 잘못)

"용량 무한으로 주면 무제약 9,958.2 가 나와야 한다" 는 롤링으로는 불가능하다.
실제 10,078.2 (+120kg, 1.2%). 회계가 틀린 게 아니라 결정 규칙이 다르다.
기존은 제출 시점 예측 한 번으로 전 윈도우 평가,
롤링은 매 슬롯 최신 예측으로 재평가한다. 같은 값이 나올 수 없다.

** C7 정정: 모든 작업을 제출 시각에 그대로 실행시키면
비교군2 12,609.8 kg 이 나와야 한다. **

그러면 결정 규칙을 배제하고 회계만 검증된다.

논문에 명시해야 할 설계 차이 (ac 지적)

롤링은 큐에 남은 작업을 매 슬롯 최신 예측으로 다시 평가한다.
공식(식 11)은 그대로지만 입력을 매 슬롯 갱신하는 건 실질적 설계 변경이다.
"미래를 안다고 가정하지 않는다" 는 강점으로 쓸 수 있지만 반드시 밝혀야 한다.

검증 스크립트: scratchpad/verify_cap.py

코드의 자기 보고(peak, hours_over)를 쓰지 않고

결과(scheduled_start, duration, region)에서 연속 시각 이벤트 스왑으로 다시 센다.

[26] 용량 인지 최종 검증 통과 – 2026-09-16

장부 버그 수정 후 재검증. 독립 재계산으로 확인.

새 헤드라인

기준	29,225.6 kg		
공간 이동만	12,609.8 kg	56.85%	
+시간 무제한	9,958.2 kg	65.93%	실현 불가. 반사실 상한
+시간 온라인	10,830.4 kg	62.94%	<- 새 결과
+시간 사후강제	11,873.0 kg	59.37%	순진한 하한 (베이스라인으로 재라벨)

시간 이동의 한계 기여 (기준 대비)

무제한	9.07%p	
온라인	6.09%p	<- 새 결과
사후강제	2.52%p	

구간 회수율 54.4%

논문 서술이 이렇게 바뀐다

(전) "실현 가능한 절감은 59.37% 와 65.93% 사이에 있다"
(후) "온라인 용량 인지 시간 이동으로 62.94% 를 달성한다.
무제한 65.93% 는 용량을 지키지 않은 반사실 상한이고,
사후 강제 59.37% 는 재배치를 하지 않는 순진한 베이스라인이다."
구간이 값 하나로 확정된다. 한계 1번과 향후과제 1번이 동시에 지워진다.

용량 초과가 얼마나 줄었나 (§ 6.4 정의로 독립 재계산)

	무제한	온라인	감소
최대 동시	41건	19건	-54%
FR 상한 초과 시간	622.4h	23h	-96%
CAL 상한 초과 시간	866.9h	12h	-98.6%
상한 위반 배경	(미측정)	662건	전체의 0.45%

남은 662건 중 411건이 마감 임박 강제 편입이다.
나머지 251건은 지연 예산이 슬롯보다 짧아 미룰 수 없는 작업이다.

즉 남은 초과분은 전부 "미룰 수 없는 작업" 때문이다. 알고리즘 결함이 아니다.

검증 통과 현황

C1 마감 위반 0	통과
C2 처리 146,000 전량	통과
C3 구간 안 (10,830.4)	통과
C4 장부 = 독립 재계산	통과 (이전엔 불일치)
C6 지연 51,273 <= 58,288	통과

C7 회계 = 비교군2 12,609.8 통과 오차 0.0 kg
C5 -> 기준 자체가 틀렸음. 아래 참고

남은 정리 항목 3개

1. C5 기준이 틀렸다 (1b 잘못). 그리고 논문 본문이 바뀌어야 한다.
k=3,4,5 중 544건이 지연됐는데 C1 이 통과하므로 전부 예산 이내다.
제출 15:54 에 예산 10분인 작업이 16:00 에 도는 것은 6분 지연이고 정상이다.
-> 문단 568 "지연 예산이 1시간보다 짧은 k=3,4,5는 시간 이동의 대상이 되지 않는다" 가 롤링에서는 거짓이 된다. 고쳐야 한다.
그리고 § 6.4 의 분모 58,288 도 다시 봐야 한다.
2. 용량 위반의 정의가 둘이다. 하나로 통일할 것.
코드 1,861건 (슬롯마다 |F| - a 를 누적)
독립계산 662건 (시작 시각에 실제 동시 실행이 12 이상이었던 배경)
-> 논문은 § 6.4 정의와 일관된 후자를 쓸 것. 코드는 보수적 상한이다.
3. 즉시 실행 경로가 용량 검사를 받지 않는다.
미룰 수 없는 작업이므로 막을 수 없다. 다만 강제 편입으로 집계해야 한다.
지금은 forced=False 로 기록돼 251건이 어디에도 안 세어진다.

실행 시간 1년치 146,000 작업 8초. 재실행 부담 없음.

[27] 2026-09-16 종료 — 오늘 끝난 것과 다음

오늘 끝난 것

- | | | |
|------------------|------|-----------------------------------|
| 1. alpha 표 완성 | [17] | 숫자 확보, 표2와 일치. 10초 |
| 2. 그림 1 수정 | | 곡선 vs 값 1개 대비 + 각주. 451x256 |
| 3. 용량 인지 구현 + 검증 | [26] | 62.94% 달성. 구간이 값 하나로 확정 |
| 4. 공식 명세 작성 | | 공식.txt (규약 · 증명) + 공식.tex (Word용) |
| 5. 백업 | | paper/백업/ 와 백업_코드/ 둘 다 |

가장 큰 성과

헤드라인이 "59.37~65.93% 사이" 에서 "62.94%" 로 확정됐다.
한계 1번과 향후과제 1번이 동시에 지워진다.
용량 초과: 최대 동시 41 -> 19, 초과 시간 867h -> 12h (CAL), 622h -> 23h (FR)

일하는 방식이 효과가 있었다

구현(82) / 코드리뷰(ac) / 숫자검증(1b) 을 분리했더니
- ac 가 코드에서 찾은 장부 문제를 1b 가 숫자로 확인 (73배 불일치)
- 1b 가 짠 코드의 버그를 ac 가 찾음
- 1b 의 검증 기준 두 개(C5, C7)가 잘못 설계됐음이 드러남
혼자 했으면 다 놓쳤을 것들이다. 이 방식을 계속 쓸 것.

다음에 할 것 — 순서대로

- 실험 없이 되는 것
1. 없는 인용문 3개 수정 문단 84, 100, 151, 335 **최우선**
 2. 수식 15개를 Word 수식 개체로 지금 0개다. 한 시간쯤
 3. 초록 쓰기 국문 300~500자 + 영문
 4. 기여 순서 바꾸기 무류점 alpha 를 1번으로
 5. Caspian 관련연구 추가 2.3절 + 구조차용 문장
 6. alpha 표 + 파레토 그림 삽입
 7. 기상 예보 누출 한계 추가

용량 인지 반영 (숫자가 이미 나왔으므로 서술만)

9 표2 에 4행 추가 (온라인 62.94%)

10 6.4절 성격 전환 "안 한 것" -> "대조군으로 설계한 것"

11 문단 568 수정 $k=3,4,5$ 도 예산 내에서 움직인다

12 3.6절 신설 식 (13)(14)(15) + Algorithm 1 + 증명

코드 정리 (작음)

13 용량 위반 정의 통일 코드 1,861 vs 독립계산 662

14 즉시실행 경로 집계 누락 251건이 어디에도 안 세어짐

15 B3 시간 이동 단독 검증 home_region=True 로 이미 있음. 확인만

나중에

16 용량 스위프 재현 스크립트 복원

17 솔버 시간 측정 무류점 alpha 의 계산 비용. 논문에 한 줄도 없다

작업 문서 4개

정리.txt 무엇이 정해졌고 왜 그런지 (근거 · 수치 · 결정)

목차.txt 각 절에 무엇을 쓸지 (집필용)

공식.txt 기호 · 규약 · 증명. 코드와 다르면 이 파일이 기준

공식.tex Word 에 붙여넣을 수식 소스